DataLab: A Unified Platform for LLM-Powered Business Intelligence

Luoxuan Weng[†‡], Yinghao Tang[†], Yingchaojie Feng[†], Zhuo Chang[§‡], Ruiqin Chen[‡], Haozhe Feng[‡], Chen Hou[‡], Danqing Huang[‡], Yang Li[‡], Huaming Rao[‡], Haonan Wang[‡], Canshi Wei[‡], Xiaofeng Yang[‡], Yuhui Zhang[‡], Yifeng Zheng[‡], Xiuqi Huang[†], Minfeng Zhu[‖], Yuxin Ma[¶], Bin Cui[§], Peng Chen[‡✉], Wei Chen[†✉]

[†]*State Key Lab of CAD&CG, Zhejiang University*, [‡]*Tencent Inc.*, [‖]*Zhejiang University*,
[¶]*Southern University of Science and Technology*, [§]*School of Computer Science, Peking University*
[†]{lukeweng, yinghaotang, fycj, huangxiuqi, chenvis}@zju.edu.cn, [‡]{ruiqinchen, aidenhzfeng, rickhou, daisyqhuang thomasyngli, huamingrao, nanthanwang, caydenwei, felixxfyang, yukiyhzhang, yifengzheng, pengchen}@tencent.com, [‖]minfeng_zhu@zju.edu.cn, [¶]mayx@sustech.edu.cn, [§]{z.chang, bin.cui}@pku.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00326

***Abstract*—Business intelligence (BI) transforms large volumes of data within modern organizations into actionable insights for informed decision-making. Recently, large language model (LLM)-based agents have streamlined the BI workflow by automatically performing task planning, reasoning, and actions in executable environments based on natural language (NL) queries. However, existing approaches primarily focus on individual BI tasks such as NL2SQL and NL2VIS. The fragmentation of tasks across different data roles and tools lead to inefficiencies and potential errors due to the iterative and collaborative nature of BI. In this paper, we introduce DataLab, a *unified* BI platform that integrates a *one-stop* LLM-based agent framework with an *augmented* computational notebook interface. DataLab supports various BI tasks for different data roles in data preparation, analysis, and visualization by seamlessly combining LLM assistance with user customization within a *single* environment. To achieve this unification, we design a domain knowledge incorporation module tailored for enterprise-specific BI tasks, an inter-agent communication mechanism to facilitate information sharing across the BI workflow, and a cell-based context management strategy to enhance context utilization efficiency in BI notebooks. Extensive experiments demonstrate that DataLab achieves state-of-the-art performance on various BI tasks across popular research benchmarks. Moreover, DataLab maintains high effectiveness and efficiency on real-world datasets from Tencent, achieving up to a 58.58% increase in accuracy and a 61.65% reduction in token cost on enterprise-specific BI tasks.**

***Index Terms*—Business Intelligence, LLM, Data Analysis**

I. Introduction

Business intelligence (BI) aims to transform large volumes of data into actionable insights for informed decision-making [1]. A typical BI workflow includes multiple stages such as data preparation, analysis, and visualization. It requires the collaboration of data engineers, scientists, and analysts using various specialized tools (*e.g.*, Visual Studio Code, Power BI, Tableau), which can be highly tedious and time-consuming [2]. Therefore, modern organizations require advanced techniques to automate and optimize this workflow.

This work is done during Luoxuan Weng's internship at Tencent TEG Big Data Platform. We thank the great support from Tencent Cloud ChatBI and Tencent Ad DataWarehouse Team. This work is supported by the National Natural Science Foundation of China (62132017, 62302435, 62421003) and Zhejiang Provincial Natural Science Foundation of China (LD24F020011). Wei Chen and Peng Chen are the corresponding authors.

Recent advancements in autonomous agents powered by large language models (LLMs) [3] offer the potential to streamline the BI workflow. By receiving instructions in natural language (NL), LLM-based agents can perform task planning, reasoning, and actions in executable environments. This can significantly reduce the complexity of many BI tasks, such as code generation [4], text-to-visualization translation [5], and automated insight discovery [6].

However, previous works on LLM-based agents for BI primarily focus on individual tasks or stages without considering the BI workflow as a whole. The separation of BI tasks across different data roles and tools impedes seamless information flow and insight exchange, adding to communication costs, delays, and errors [7], [8]. For example, data analysts using GUI-based platforms (*e.g.*, Power BI) often rely on data engineers working with development tools (*e.g.*, PyCharm) to prepare data for analysis or visualization. This reliance necessitates back-and-forth communication between analysts and engineers due to the iterative and collaborative nature of BI [1]. Such procedures highlight the limitations of existing fragmented and fixed agent pipelines [9]. Consequently, this leads to a significant gap among different roles, tasks, and tools, which hinders timely and informed decision-making.

To bridge this gap, we aim to unify the BI workflow with a *one-stop* LLM-based agent framework in a *single* environment that satisfies the requirements of various data roles. However, achieving this unification in practical enterprise settings is non-trivial due to the following challenges:

C1: Lack of domain knowledge incorporation. Existing studies usually leverage clean and synthesized research benchmarks to build and evaluate agents [10]. However, BI tasks typically involve large and dirty real-world datasets with many ambiguities [11]. For example, column names in business data tables may have unclear semantic meanings [12], and user queries often include enterprise-specific jargon [10]. To mitigate these issues, incorporating extensive domain knowledge is essential to enhance agents' understanding of input data and improve their performance on practical BI tasks. While some approaches adopt fine-tuning [13] or continued pre-training [14] to augment agents' domain-specific capa-

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00326

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

bilities, acquiring the necessary large and up-to-date training data corpora remains challenging in BI scenarios due to their complexity and dynamic nature. Other approaches (*e.g.*, Chat2Data [15]) require users to manually integrate domain knowledge through external documents or customized knowledge bases, which is highly inefficient and inconvenient.

C2: Insufficient information sharing across tasks. Different tasks are typically managed by corresponding LLM-based agents to achieve optimal performance [16]. As a complex BI query may encompass multiple tasks, information sharing among the involved agents is critical. For example, the data retrieved by a *SQL writing agent* must be accurately conveyed to a *chart generation agent*. Therefore, an effective and efficient inter-agent communication mechanism is essential to align their understanding of the overall analysis goals, current data context, and executed actions. However, many existing multi-agent frameworks, such as ChatDev [17] and CAMEL [18], rely on unstructured natural language for communication, which can lead to distortions due to the inherent vagueness and redundancy of NL [19]. Consequently, they are inadequate for handling the complexity of BI tasks, which commonly involve diverse information types (*e.g.*, data, charts, texts).

C3: Demand for adaptive LLM context management. LLM-based agents depend on their *context windows* (*i.e.*, limited input tokens for NL understanding, reasoning, and generation) to complete tasks. Necessary contexts must be provided to ensure a successful and seamless workflow. Meanwhile, in a unified BI platform, vast amounts of multi-modal contexts (*e.g.*, code snippets and their execution results, charts and their specifications) are intertwined and often relate to diverse data tables. Obviously, only relevant portions of these contexts are pertinent to specific tasks and should be selectively provided to the agents [20]. In contrast, existing works either focus on single-modal contexts tied to specific tasks [21] or indiscriminately provide all contexts [16], neither of which meets the demands of a unified BI platform. Thus, adaptive context management tailored for BI scenarios that considers prior states and current user needs is crucial for maintaining system efficiency and cost-effectiveness.

In this paper, we introduce DataLab, a *unified* environment that supports various data tasks throughout the BI workflow, thereby serving different data roles whether they use Markdown, SQL, Python, or no-code, all within a *single* computational notebook. We use notebooks as the foundational system due to their popularity in data science [7] and their iterative nature for the BI workflow [1]. DataLab adopts an LLM-based agent framework to integrate LLM assistance seamlessly, and a notebook interface to enable user customization flexibly.

To improve agents' performance on enterprise-specific BI tasks (for **C1**), we develop a *Domain Knowledge Incorporation* module, a systematic approach for automated knowledge generation, organization, and utilization. It leverages data processing scripts (*e.g.*, Python code, SQL queries) within the enterprise to extract knowledge associated with databases/tables/columns, thereby uncovering their common usage patterns. This module overcomes the practical challenge of manually integrating external knowledge bases for BI.

To facilitate information sharing across different tasks (for **C2**), we design an *Inter-Agent Communication* module, a structured mechanism that goes beyond pure NL to enhance the information representation capabilities of agents. It also formulates the information sharing process among agents with a finite state machine (FSM) for a more controlled and efficient flow of communication. This module addresses the challenge of effectively and efficiently sharing multi-modal information generated by various agents throughout the BI workflow.

To manage LLM contexts within multi-modal notebooks (for **C3**), we propose a *Cell-based Context Management* module that represents inter-cell dependencies using directed acyclic graphs (DAGs). These dependency graphs are dynamically updated in response to user modifications, enabling the adaptive selection of pertinent contexts based on specific task requirements. This module resolves the challenge of enhancing agents' context utilization efficiency in BI scenarios.

Compared to existing works, DataLab stands out due to four benefits: (1) It delivers satisfactory performance across various BI tasks, rather than focusing on individual tasks; (2) It is well-suited for real-world applications, not just research benchmarks; (3) It offers a unified platform to satisfy different user requirements, rather than catering to a single data role; and (4) It integrates LLM assistance with user customization, instead of relying solely on end-to-end result generation.

In summary, the major contributions of our work are:

- We present DataLab, a platform that unifies the BI workflow with the integration of a one-stop LLM-based agent framework and a computational notebook interface, to bridge the gap among different roles, tasks, and tools.
- We develop a systematic approach for domain knowledge incorporation to enhance LLM-based agents' performance on enterprise-specific BI tasks in practical settings.
- We introduce a structured communication mechanism to formulate the information sharing process among different agents to facilitate their cross-task performance.
- We propose an adaptive context management strategy to improve agents' context utilization abilities within computational notebooks for efficiency and cost-effectiveness.
- We extensively evaluate DataLab on both research benchmarks and real-world business datasets from Tencent, demonstrating its performance on various BI tasks. We also showcase the practical applications of DataLab at Tencent TEG's Big Data Platform through user feedback.

II. Background

A. BI Workflow

The BI workflow traditionally includes several key stages: data collection, storage, preparation, analysis, and visualization. Data Collection and Storage establish the foundation, typically supported by cloud infrastructures (*e.g.*, Tencent TCHouse-C, AWS) and specialized data applications (*e.g.*, BigQuery, Airflow) with mature ecosystem integration. Attempting to replicate these practices in alternative environ-

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

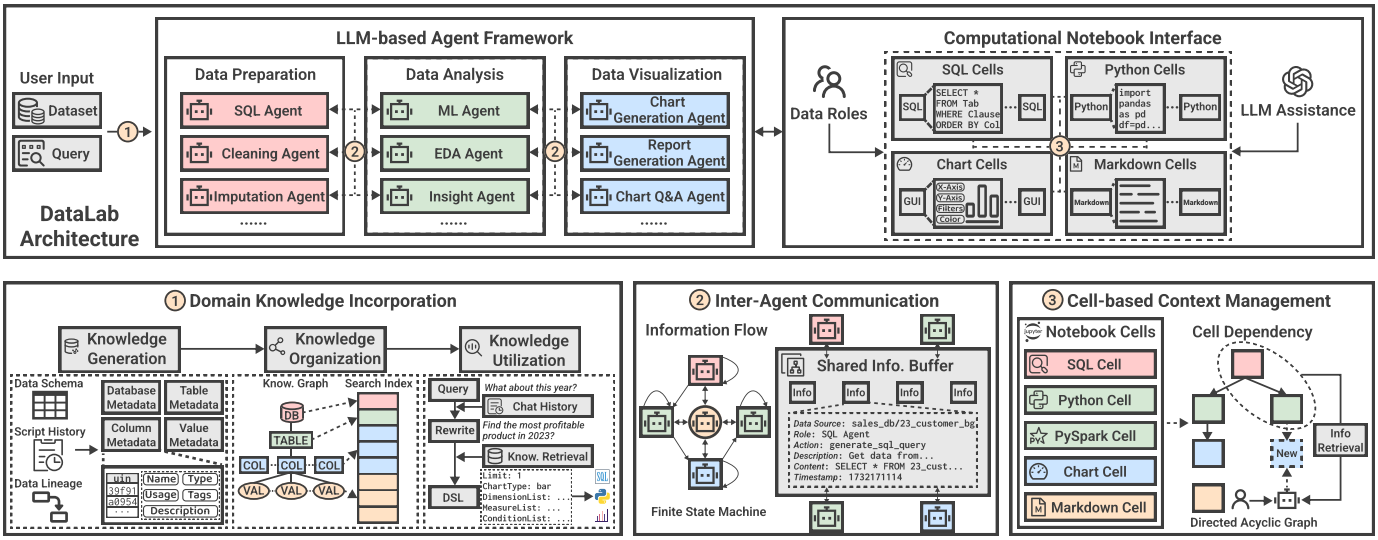


Fig. 1. Overview of DataLab and its three critical modules.

ments could introduce inefficiencies and anti-patterns. Moreover, the core value of modern BI lies in transforming *raw data* into actionable *insights* through data preparation, analysis, and visualization [1], [8]. Therefore, our work primarily focuses on the latter three stages, emphasizing *interoperability* with existing tools rather than reinventing them. For data collection and storage, we provide plugin APIs to enable seamless integration with third-party data connectors, ensuring flexibility and compatibility within the broader BI ecosystem.

Data Preparation [22] ensures data consistency, correctness, and quality. This usually includes cleaning, structuring, and enriching raw data into a format suitable for further analysis. Following preparation, **Data Analysis** [23] applies statistical and analytical techniques to extract insights, aiming to uncover patterns, trends, and correlations. Finally, **Data Visualization** [5] presents analyzed data in visual formats like charts, graphs, and dashboards, which makes complex data easier to understand and interpret for decision-makers.

The data roles involved in the BI workflow are specific to different organizations. Among them, data engineers, scientists, and analysts are usually indispensable. **Data Engineers** are primarily tasked with data preparation, constructing and administering data pipelines to ensure that data is accurately cleansed and structured for subsequent analysis. They typically use SQL and Python for data processing, and use cloud computing platforms like AWS for data storage and ingestion. **Data Scientists** engage in data preparation and analysis, applying advanced statistical and machine learning methodologies to extract insights and forecast trends from intricate datasets. They are familiar with Python/R and popular data science libraries like `Pandas`. **Data Analysts** concentrate on data analysis and visualization, analyzing data to discern patterns and conveying findings through detailed visual reports and dashboards. They use SQL to query data, and rely on BI platforms like Tableau to perform and share their analyses.

In modern enterprises, a complex BI workflow requires the collaboration of multiple data roles across various stages. The current fragmentation of tools for data preparation, analysis, and visualization introduces frictions and delays in timely decision-making. Therefore, an integrated and unified platform can serve as a shared environment for distinct user groups, facilitating the efficiency, transparency, and productivity of BI.

B. LLM-based Agents for BI

LLM-based agents are autonomous systems powered by LLMs that can perceive environments, execute tasks, make decisions, and interact with users in complex contexts [3]. These agents comprise profiling, memory, planning, and action modules, which respectively define the agent's role, facilitate operations in dynamic environments through recall and future action planning, and convert decisions into outputs [24]. In BI scenarios, agents receive users' NL queries and then perform data preparation, analysis, and visualization. By interpreting execution results, they can complete many BI tasks. For example, data preparation involves tasks like **NL2SQL** [25] and **NL2DSCode** [4], while data analysis and visualization rely on **NL2Insight** [26] and **NL2VIS** [5], respectively.

However, most existing LLM-based agents are limited to individual tasks and do not meet the diverse user requirements of a complex BI workflow. Moreover, they often neglect the integration of enterprise-specific knowledge, resulting in unsatisfactory performance on proprietary business datasets. This lack of generalizability and customizability highlights the need for a structured and adaptive agent framework for BI.

III. Overview

Architecture Overview. As illustrated in Figure 1, DataLab consists of two primary components: (1) *LLM-based Agent Framework* and (2) *Computational Notebook Interface*.

- **LLM-based Agent Framework.** In DataLab, multiple agents are designed for different BI tasks based on user requirements. To achieve this, we first identify several common BI procedures and abstract them into *data tools* that can be called upon by agents during inference. Example tools include a Python sandbox for code execution and a Vega-Lite environment for visualization rendering. Accompanied

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

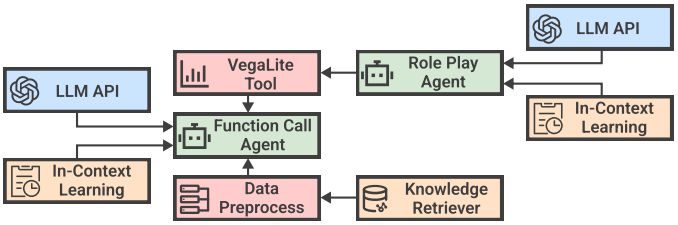


Fig. 2. Example agent workflow for NL2VIS.

by other auxiliary components like memory modules, each BI agent is represented as a DAG for high flexibility and easy extensibility. Within the DAG, nodes depict reusable components (*e.g.*, LLM APIs, tools) and edges depict their connections (*e.g.*, file transfer across tools). Figure 2 illustrates an example agent workflow for NL2VIS. Additionally, we add a *proxy agent* to the framework, which serves as a hub to directly interact with users and allocate tasks to each specialized agent based on user queries. Compared to existing approaches that primarily focus on individual tasks (*e.g.*, NL2VIS [27], NL2SQL [21]), DataLab's agent framework supports a wide array of tasks for various data roles across the BI workflow through multi-agent collaboration.

- **Computational Notebook Interface.** DataLab's notebook interface (Figure 3) serves as a unified, interactive, and collaborative environment for different data roles to complete their specialized tasks. To achieve this, we augment JupyterLab (a widely used notebook interface) to support (1) multi-language cells and (2) on-the-fly LLM assistance. First, DataLab wrangles SQL, Python/PySpark, Markdown, and Chart cells altogether, allowing both technical and non-technical users to easily adopt their familiar workflows on the notebook. Going beyond traditional notebooks that only support Python and Markdown, DataLab notebooks directly connect to backend databases for SQL query execution, and feature GUI-based dashboards [28] similar to Tableau for visualization authoring. Second, we integrate our LLM-based agent framework seamlessly into each notebook cell. Users can get LLM assistance both at notebook- and cell-level. Specifically, users toggle an input box and type their analytic queries, which are then processed by the agents in our framework. These agents can create new cells or modify existing ones in the notebook. Users can subsequently examine the results and make further customizations flexibly. Compared to existing end-to-end approaches (*e.g.*, CHESS [29], LIDA [30]), DataLab's notebook interface enables flexible user intervention to adapt LLM-generated results to real-world BI scenarios.

DataLab Workflow. Upon receiving an NL query and the associated dataset, DataLab analyzes the dataset and interprets the query, incorporating domain knowledge (Figure 1①) before feeding them into LLMs. Then, DataLab leverages various agents to complete the task, which may involve information sharing with each other through a structured communication mechanism (Figure 1②). Subsequently, the corresponding result will be presented in the notebook. Users can either accept, edit, or reject the result and continues the BI workflow.

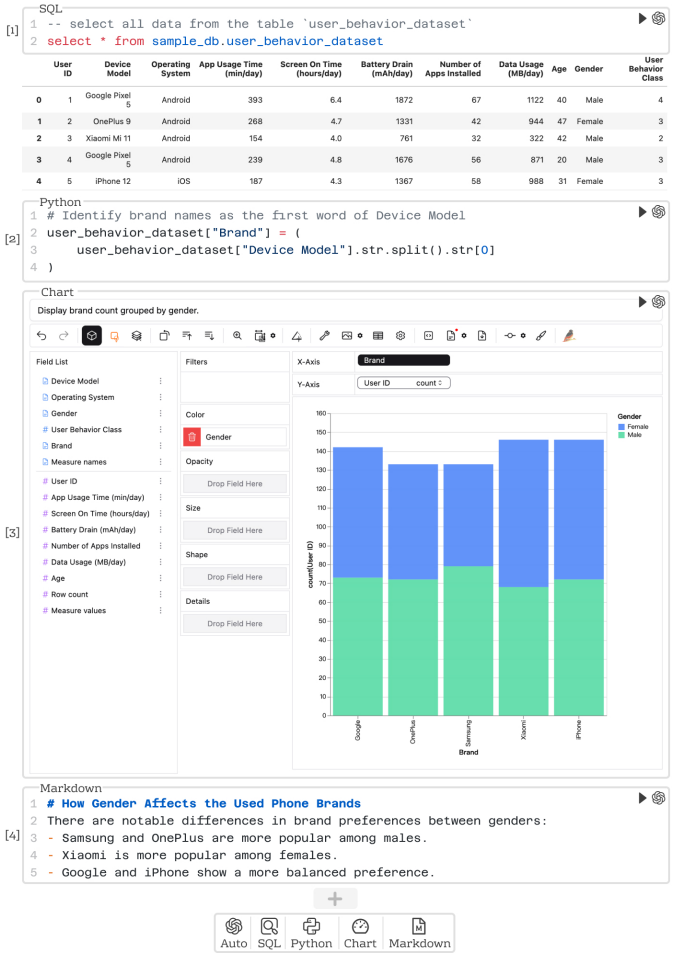


Fig. 3. The notebook interface of DataLab.

Meanwhile, a context management strategy (Figure 1③) automatically generates and maintains cell dependencies within the notebook to promote further agent calls. Next, we provide an overview of DataLab's three critical modules.

- **Domain Knowledge Incorporation.** This module takes a data table's schema, its associated script history (*e.g.*, SQL queries, Python code), and its data lineage information as input. Specifically, the schema provides a basic overview of the table and its columns, including their names and types. The associated data processing scripts, which are created by professionals and executed every day within the organization, reflect the semantic meanings and common usage patterns of the table and its columns. And the data lineage information [31], which reveals interrelationships among distinct tables and columns across the organization, can serve as an auxiliary resource for domain knowledge extraction. Based on the input, the module leverages LLMs to automatically generate the *knowledge components* (*e.g.*, descriptions, usages) of databases, tables, columns, and certain values. These knowledge components are then organized in a knowledge graph to facilitate further retrieval and utilization, which translate ambiguous user queries into structured domain-specific languages (DSLs) for improved agent performance on enterprise-specific BI tasks.
- **Inter-Agent Communication.** This module formulates the

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

information flow process among different agents as an FSM to enable more control over their communications, with nodes representing agents and edges representing inter-agent information transition directions. Upon task completion, each agent's outputs are formatted into structured *information units* [19], comprising key characteristics such as the associated table's identifier and a concise description of the actions performed. The module also maintains a shared information buffer to proactively exchange information based on the FSM to improve communication efficiency.

- **Cell-based Context Management.** This module identifies cell dependencies within a notebook based on variable references and constructs a DAG, where nodes represent cells and edges denote their dependencies. Notably, data variables in Python or SQL cells are meticulously tracked, such as `DataFrames` and `SELECTs`. Given a user query, the module traverses the DAG to locate relevant cells, performs pruning based on task types, and retrieves information from the shared buffer. Then, the original cells and their corresponding information units are fed to the proxy agent as necessary contexts to facilitate task completion.

IV. Domain Knowledge Incorporation

In this section, we introduce DataLab's *Domain Knowledge Incorporation* module, which encompasses three stages, namely knowledge generation, organization, and utilization.

A. Knowledge Generation

Ambiguities are pervasive in real-world BI scenarios, manifesting both in the underlying databases and users' NL queries. For example, consider the query, *'show me the income of TencentBI this year'*, which involves three columns: *'prod_class4_name'*, *'shouldincome_after'*, and *'ftime'*. The semantic relationships between these column names and the user's request are often vague, leading to LLMs' suboptimal performance on such tasks. To mitigate these issues, existing approaches integrate table schema [5] into prompts and adopt retrieval-augmented generation (RAG) [10] to improve LLMs' domain-specific capabilities. We categorize three types of domain knowledge commonly utilized for BI tasks:

- **Metadata:** Information about data structure and attributes, such as table and column names, types, descriptions, and common usage patterns.
- **Business Logic:** Rules and processes that dictate how data is used and interpreted within the business.
- **Jargon:** Specialized terminologies and acronyms specific to the industry or organization.

Many existing works assume that such knowledge can be manually constructed and integrated by domain experts. However, in large organizations, enormous numbers of data tables are commonly involved, making manual curation and maintenance highly impractical. To overcome this limitation, we first conducted an extensive investigation at Tencent. It was observed that, while 85% of the tables lack comprehensive metadata, they are predominantly linked to various SQL or Python scripts utilized for data processing. These scripts reveal common usage patterns within practical business contexts. Additionally, for those tables lacking adequate processing scripts, data lineage information, which elucidates their connections to other tables or columns throughout the organization, provides an alternative resource for metadata imputation. Therefore, inspired by LLMs' exceptional code understanding and reasoning abilities, we propose an LLM-based knowledge generation approach (Algorithm 1) that leverages script history to abstract and summarize knowledge components through meticulously-designed prompting techniques. This automated approach comprises a Map-Reduce process with a self-calibration mechanism [32] to generate high-quality knowledge for databases, tables, and columns.

Algorithm 1 LLM-based Knowledge Generation

Input: Schema $\mathcal{S}$, Script History $\mathcal{H}$,
Lineage Information $\mathcal{L}$, Score Threshold $\mathcal{T}$
Output: Database/Table/Column Knowledge $\mathcal{D}$, $\mathcal{T}$, $\mathcal{C}$

$\mathcal{H} \leftarrow$ preprocess($\mathcal{H}$) *// Duplicated/Similar script filtering*
map_res $\leftarrow$ [ ]
for each historical script $h_i \in \mathcal{H}$ **do**
  while $s_i < \mathcal{T}$ **do**
    $d_i, t_i, c_i \leftarrow \text{LLM}(h_i, \mathcal{S}, \mathcal{L})$ *// Knowledge generation*
    $s_i \leftarrow \text{LLM}(d_i, t_i, c_i)$ *// Self-calibration*
  end while
  map_res.append($[d_i, t_i, c_i]$)
end for
$\mathcal{D}, \mathcal{T}, \mathcal{C} \leftarrow \text{LLM}(\text{map_res}, \mathcal{S}, \mathcal{L})$ *// Knowledge synthesis*
return $\mathcal{D}, \mathcal{T}, \mathcal{C}$

Knowledge Components. Considering the previously defined knowledge categories, metadata and business logic can be deduced from data processing scripts, as both SQL queries and Python code support data manipulation operations like filtering and aggregation. Business logic is essential for computing *derived columns* which, though absent in the original table, hold significant value in business contexts. In contrast, jargon primarily exists in user queries or organization wikis (*i.e.*, documents), necessitating enterprise-specific glossaries for management and application. The *knowledge components* that our automated approach can generate are outlined below:

- **Database Level:** *description, usage, tags.*
- **Table Level:** *description, usage, organization, key column names, key derived attribute names, tags.*
- **Column Level:** *description, usage, type, tags, derived column information (name, description, usage, calculation logic, related columns, tags).*

These knowledge components are structured using JSON formats to improve LLMs' generation performance.

Map Phase. Given a data table, we take its schema $\mathcal{S}$, its script history $\mathcal{H}$, and its lineage information $\mathcal{L}$ as input. During the map phase, each distinct historical script h_i is individually processed using an LLM as the mapping model to produce corresponding knowledge components. The LLM is prompted to carefully analyze the script's semantic content and logical structure, aiming to extract critical information relevant to the specific business context. To mitigate LLMs' hallucination issues, focus is restricted to the involved databases, tables, and columns. This process results in the generation of a set of knowledge components associated with the script h_i.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

Self-Calibration. Within each iteration of the map phase, we integrate a self-calibration mechanism that leverages LLMs' self-reflection abilities [33] to evaluate the intermediate results using a numerical score ranging from 1 to 5. Specifically, we instruct the LLM to consider multiple aspects of the knowledge components (*e.g.*, correctness, comprehensiveness, clarity) and provide several manually crafted in-context examples to demonstrate the scoring criteria. Should the rating score s_i fall below the predefined threshold $\mathcal{T}$, the knowledge generation process must be repeated. Therefore, this feedback loop ensures the generation quality of each iteration.

Reduce Phase. During the reduce phase, we aim to synthesize the individual results derived from each historical script to produce the final sets of knowledge components $\mathcal{D}$, $\mathcal{T}$, and $\mathcal{C}$ for the involved database, table, and columns, respectively. The LLM is instructed to meticulously scrutinize, aggregate, and summarize the information from all separate results to ensure a consistent and conflict-free collective result.

For each data table at Tencent, we execute the above Map-Reduce process to generate a comprehensive and high-quality set of knowledge components, which can significantly benefit many downstream BI tasks.

B. Knowledge Organization

We employ a knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ to systematically organize the knowledge generated by our automated approach (*i.e.*, metadata and business logic) and the manually crafted enterprise-specific glossaries (*i.e.*, jargon).

As depicted in Figure 4, the knowledge graph adopts a tree-based structure for knowledge organization. The nodes $\{\mathcal{V}\}$ are structured into five primary types: *database*, *table*, *column*, *value*, and *jargon*, each comprising various components (*e.g.*, *description*, *usage*) and uniquely identified by a *name*. To address the common challenge of terminological inconsistencies in user queries (*e.g.*, synonyms, acronyms), an additional node type, *alias*, has been introduced. This node type contains alternative terms associated with the official *name* of other node types. It is predetermined based on enterprise-specific glossaries and can be dynamically updated in real-world applications. The relationships between these nodes are represented by edges $\{\mathcal{E}\}$, which delineate both *logical relationships* among the primary node types and *associative relationships* between *alias* nodes and other primary nodes.

To facilitate efficient knowledge retrieval, we develop a task-aware indexing mechanism for graph nodes, utilizing Elasticsearch [34] for full-text search and StarRocks [35] for embedding search. This supports both lexical and semantic matching of knowledge nodes in response to user queries. The indexing structure is designed as triplets ($\{\text{name}, \text{content}, \text{tag}\}$), where the *content* field is a concatenation of knowledge components specified based on the various requirements of downstream tasks. For instance, some tasks necessitate the *calculation logics* while others only need *descriptions* for successful completion. By dynamically selecting the appropriate index, we ensure that knowledge retrieval is both efficient and effective.

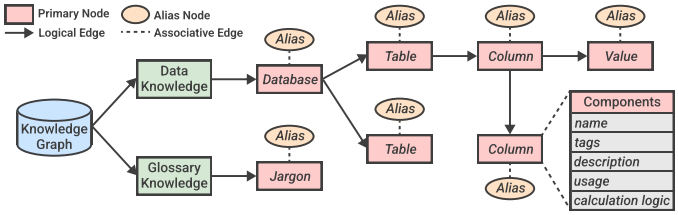


Fig. 4. Structure of the knowledge graph.

C. Knowledge Utilization

Algorithm 2 Knowledge Retrieval

Input: User Query $\mathcal{Q}$, Knowledge Graph $\mathcal{G}$
Output: Knowledge Nodes $\mathcal{V}_\mathcal{Q}$

$\mathcal{V}_\mathcal{Q} \leftarrow \emptyset$
Coarse-Grained Retrieval:
$\mathcal{V}_\mathcal{Q} \leftarrow \text{lex_search}(\mathcal{Q}, \mathcal{G}) + \text{sem_search}(\mathcal{Q}, \mathcal{G})$
Fine-Grained Ordering:
for each node $v_i \in \mathcal{V}_\mathcal{Q}$ **do**
  if v_i.type == alias **then**
    $v_i \leftarrow \text{backtrack}(v_i)$ *// Backtrack to a primary node*
  end if
  // Compute a weighted matching score
  $\text{score}_i \leftarrow \omega_1 \cdot \text{lex_eval}(\mathcal{Q}, v_i) + \omega_2 \cdot \text{sem_eval}(\mathcal{Q}, v_i) + \omega_3 \cdot \text{LLM_eval}(\mathcal{Q}, v_i)$
end for
$\mathcal{V}_\mathcal{Q}.\text{sort}(\text{score}_i)$ *// Rank by matching score*
return $\mathcal{V}_\mathcal{Q}.\text{topK}$

As shown in Figure 1①, given a user query $\mathcal{Q}$, we initially rewrite it to enhance clarity and detail. We then retrieve its relevant knowledge from the knowledge graph $\mathcal{G}$. Following this, the query is translated into a DSL specification, facilitating downstream tasks like NL2SQL and NL2VIS.

Query Rewrite. In addition to ambiguities, user queries can also be incomplete or underspecified, especially in multi-round interactions. For example, queries might omit prior context with phrases like *'what about'*. To ensure effective knowledge retrieval, the original query is enhanced and rewritten into a clearer and more detailed form, incorporating relevant prior information when available. Notably, temporal references (*e.g.*, *'last year'*) are also standardized based on the current time.

Knowledge Retrieval. To enhance LLMs' domain specific performance by integrating relevant knowledge into their context alongside the query, the selection and ordering of knowledge nodes from the knowledge graph are crucial. We employ a coarse-to-fine approach (Algorithm 2) to ensure comprehensive and precise knowledge retrieval.

- **Coarse-Grained Retrieval:** We perform lexical and semantic searches to retrieve a coarse set of knowledge nodes via token matching and embedding similarity between the query and each node's indexing triplet. We set a rather loose threshold to maximize recall. For *alias* nodes, we trace back to identify the nearest primary nodes (*i.e.*, database/table/column/value/jargon nodes).
- **Fine-Grained Ordering:** To prioritize the retrieved nodes, we implement a scoring mechanism that calculates a weighted matching score for each node, assessing its relevance to the query. This involves a three-stage evaluation: token-based (*i.e.*, lexicon), embedding-based (*i.e.*, semantics), and LLM-based (*i.e.*, overall relevance) [36]. Each

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

stage yields a normalized score, with specific calculation methods and weights tailored to different BI tasks. The final set of knowledge nodes $\mathcal{V}_\mathcal{Q}$ is determined by sorting the initial node set according to these scores and selecting the top-K nodes, where K is set to a relatively large value to ensure a comprehensive coverage.

DSL Translation. The final step translates the query into a DSL specification, a common routine in BI scenarios [10]. This JSON structure specifies the relevant data and processing requirements, including fields such as *'MeasureList'* (*i.e.*, numerical columns), *'DimensionList'* (*i.e.*, categorical columns), and *'ConditionList'* (*i.e.*, filters). We prompt an LLM for DSL translation, providing detailed instructions and in-context examples to improve its performance. The generated DSL specification is validated using JSON Schema [37] to ensure syntactic and semantic correctness. This specification can then be directly converted to high-level languages like SQL and Vega-Lite based on predefined rules, or used to enhance free-form code generation for complex tasks like NL2Insight, thereby facilitating LLMs' performance in business settings.

We also introduce a fallback strategy to address scenarios where relevant knowledge is scarce, especially for in-the-wild tables. Specifically, we develop a **Data Profiling** module that systematically extracts information from the table. This module consists of two stages: (1) heuristics-based analysis, which identifies and calculates each column's name, data type (*e.g.*, *float*, *string*), basic statistics (*e.g.*, *min*, *max*), and a random sample list, and (2) LLM-based interpretation, which feeds the extracted information to an LLM to generate a semantic description of each column and the overall table. Together, these stages produce a comprehensive summary of the table, aiding in the DSL translation process.

V. Inter-Agent Communication

In this section, we introduce DataLab's *Inter-Agent Communication* module, which facilitates efficient communication among multiple agents to complete complex BI tasks. These agents are created and optimized beforehand through DAG-based workflows (Section III) to meet different user needs. For example, a *SQL writing agent* is specialized for NL2SQL tasks and a *chart generation agent* is for NL2VIS tasks.

Workflow. As shown in Figure 5, upon receiving a user query, the proxy agent initiates an analysis to formulate an execution plan (defined by an FSM), which comprises multiple subtasks allocated to various agents (*Steps 1-2*). It then dynamically manages the communication among involved agents based on task progression by retrieving information from a shared buffer and forwarding it to the agents to support subtask execution (*Steps 5-6*). Upon completion of the subtasks, the proxy agent stores the agents' outputs in the buffer (*Steps 3-4*). Finally, once all subtasks are completed, the proxy agent generates a final answer and returns it to the user (*Step 7*).

Information Format Structure. A critical consideration in multi-agent collaboration is *what 'language' agents use to communicate*. In BI scenarios, the information exchanged among agents is diverse, encompassing types such as SQL

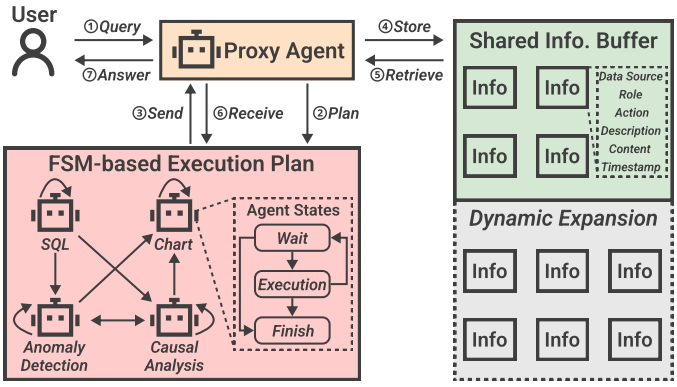


Fig. 5. Workflow of *Inter-Agent Communication*.

queries, Python code, and charts. This variety poses a significant challenge in ensuring accurate information sharing without introducing redundancy or miscommunication. Existing frameworks [17], [18] that rely on unstructured natural language for communication suffer from vagueness and inefficiencies. To address this, we design a structured information format [19] tailored to unique characteristics of BI scenarios.

An *information unit* comprises six fields: *Data Source*, *Role*, *Action*, *Description*, *Content*, and *Timestamp*. *Data Source* identifies the dataset manipulated by the agent (*e.g.*, a table identifier). *Role* indicates the identity of the agent (*e.g.*, SQL Agent). *Action* specifies the agent's behavior (*e.g.*, `generate_sql_query`). *Description* provides a summary of the agent's executed actions for the task (*e.g.*, writing a SQL query to extract specific data from the table). *Content* details the agent's output (*e.g.*, a generated SQL query). *Timestamp* records the completion time of the task. To maintain consistent communication, all agents produce messages in this format.

Information Sharing Protocol. Another key challenge to resolve is *how to ensure efficient information sharing among agents*. One extreme approach is that each agent receives information solely from its predecessor based on a plan chain [38]. While this minimizes data volume, it may cause the agent overlooking essential context (*e.g.*, background information pertinent to the task). Conversely, allowing any agent unrestricted access to the information of all others is inefficient, as irrelevant context can degrade LLMs' reasoning quality [39] and introduces additional computational overhead [40] during inference. Therefore, each agent should only receive the most relevant information for task completion.

To this end, we introduce two mechanisms to achieve this:

- **Shared Information Buffer** is a location for agents to store and retrieve information. Upon task completion, an agent deposits the produced information into this buffer via the proxy agent. This mechanism decouples information producers from consumers, thereby reducing synchronization overhead. Consequently, inter-agent communication becomes asynchronous and non-blocking, allowing producers to continue processing without awaiting the retrieval of information by consumers, and vice versa. On the other hand, to manage the substantial information volumes often associated with multi-agent collaboration, we employ

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

a dynamically growing buffering mechanism. Specifically, when the buffer reaches capacity, it automatically increases its size (*i.e.*, doubling the current capacity). Additionally, outdated information is periodically cleared. For example, if an agent's information is updated based on execution feedback, the original information is removed. This ensures that the buffer can maintain high performance while efficiently adapting to changing workloads.

- **Selective Retrieval** determines the information an agent receives from others. Inspired by the message-passing mechanism in TCP/IP [41], we design an FSM-based approach to implement this. Specifically, the proxy agent analyzes the user query and generates an FSM based on task requirements and each agent's abilities to orchestrate multi-agent information sharing, with nodes depicting agents and edges depicting information transition directions. Each agent operates within three states: *Wait, Execution,* and *Finish*. When an agent (*i.e.*, acting as a *'client'*) needs to execute a subtask, the proxy agent (*i.e.*, acting as a *'server'*) selects its relevant information from the shared buffer based on the FSM, and forwards it to the client agent. Upon receipt, the agent transitions from the *Wait* state to the *Execution* state, performs the necessary action, and produces structured information, which is then sent back to the proxy agent. The proxy agent, upon receiving the response, stores it in the buffer, and the client agent reverts to the *Wait* state. This loop continues until all subtasks are completed, at which point all agents transition to the *Finish* state and release their resources.

VI. Cell-based Context Management

In this section, we introduce DataLab's *Cell-based Context Management* module, which adaptively manages contexts in a notebook to ensure system efficiency and cost-effectiveness.

Thanks to the *Inter-Agent Communication* module (Section V), DataLab can handle complex BI tasks that require the collaboration of multiple agents. Specifically, user queries will trigger the presentation or modification of cells in the notebook, each corresponding to specific information units in the shared buffer. However, the previous module primarily aims at facilitating *individual* task completion for *single* data roles. In contrast, real-world BI scenarios often involve *multiple* data roles working on *different* tasks and collaborating within a unified notebook. This typically results in a multitude of multi-language cells (*e.g.*, SQL, Python, Chart) generated or altered by either agents or users themselves. When addressing a new user query, it is crucial to efficiently provide the proxy agent with the necessary contexts from the notebook. Simply supplying all cells and their associated information units is impractical due to inefficiency and high token costs. Therefore, we aim to identify the *minimum set* of relevant cells to minimize token usage without compromising agent performance. However, notebooks are inherently dynamic and collaborative environments, where iterative modifications reshape contextual relevance, and multi-role contributions create fragmented contexts requiring task-specific filtering. To address this challenge, we model cell dependencies within the notebook as a DAG based on variable references, and propose an adaptive context retrieval mechanism.

Algorithm 3 DAG Construction

Input: Notebook Cells $\mathcal{C}$
Output: Dependency DAG $\mathcal{G}$

```
v_hash, cell_refs ← ∅, ∅
// Identify new variables in each cell
for each cell c ∈ C do
    if c.type == Python then
        ast ← construct_ast(c)
        new_v ← find_global_variables(ast)
        v_hash[new_v] ← c
    else if c.type == SQL then
        data_v ← find_data_variable(c)
        v_hash[data_v] ← c
    end if
end for
// Find referenced cells for each cell
for each cell c ∈ C do
    external_v ← find_external_variables(c)
    cell_refs[c] ← find_ref_cells(external_v, v_hash)
end for
return G ← construct_dag(cell_refs)
```

DAG Construction. As shown in Algorithm 3, given notebook cells $\mathcal{C}$, the DAG construction process includes two steps:

- **Identify new variables.** For Python cells, we construct an abstract syntax tree (AST) to find *global* variables accessible throughout the notebook (*e.g.*, function/class definitions, package imports). We exclude local variables as they are only visible within their scope. For SQL cells, any `SELECT`'s output is stored in a data variable (*e.g.*, a `DataFrame`) for future use, and thus represents a new variable. Markdown and Chart cells do not produce variables that can be referenced elsewhere, and are thus omitted. We store the variable-cell associations using a hash table.
- **Find referenced cells.** Based on the hash table, we locate each cell's referenced cells by identifying its *external* variables defined in other cells. For Python and SQL cells, this can be easily achieved with ASTs. For Chart cells, the underlying data variable serves as the reference point. As Markdown cells do not associate with any variables, they are excluded from this step. Using the extracted cell references, a DAG of the notebook can be constructed.

The DAG keeps updating whenever a cell is created, modified, or deleted, provided that the changes pass the syntax check. This ensures real-time maintenance of cell dependencies.

Context Retrieval. Based on the cell dependency DAG and an input query, relevant cells are identified through graph traversal. This process supports queries at both *cell-level* and *notebook-level*. For cell-level queries, the search is initiated within an existing cell, allowing for the straightforward identification of all ancestral cells via the DAG. Notebook-level queries, conversely, are formulated without specifying an existing cell, which typically rely on LLMs to automatically create new cells. In such cases, we first determine the related data variable either from explicit user input or through LLM prediction. Then, we locate the initial cell c_s where the data variable is defined. To ensure through coverage, all descendant cells of c_s are considered. Additionally, since Markdown cells

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

lack references, our selection is guided by the textual similarity between cell content and the query. This process yields a comprehensive set of relevant cells $\mathcal{C}_r$ for each query.

Subsequently, $\mathcal{C}_r$ is pruned based on task types. Specifically, we employ LLMs to determine the task type contained in the query and identify the involved cell types. For example, in NL2DSCode tasks, only Python cells are considered. Accordingly, we filter out irrelevant cell types, resulting in a pruned set that constitutes the *minimum set* of relevant cells.

We then retrieve the associated information from the shared buffer for each relevant cell generated or altered by agents. The final necessary contexts for the query are determined by combining the retrieved information units with the original relevant cells, thereby providing a concise yet sufficient background for the proxy agent to understand and address the query.

VII. Experiment

We evaluate DataLab on both public research benchmarks (§A, §B) and proprietary datasets from Tencent (§C, §D, §E).

A. End-to-End Performance

To demonstrate the capabilities of DataLab as a unified BI platform, we first compare its end-to-end performance with SOTA LLM-based baselines on four typical BI tasks. For fair comparison, the baselines employed are all prompt- or agent-based methods without any pre-training or supervised fine-tuning, similar to DataLab's agent framework. We utilize **GPT-4** [51] as the foundation model for all methods.

1) Settings: The **NL2SQL** task converts natural language to SQL queries, typically marking the start of a BI workflow. We use two benchmarks (*i.e.*, Spider [42] and BIRD [43]) and compare with three baselines (*i.e.*, DAIL-SQL [25], PURPLE [21], and CHESS [29]). We use *Execution Accuracy (EX)* as the evaluation metric, which measures the execution equivalence of the generated SQL queries with ground truth.

The **NL2DSCode** task converts natural language to data science code using Python libraries like `NumPy` and `Pandas`, which happens frequently throughout the BI workflow. We use two benchmarks (*i.e.*, DS-1000 [4] and DSEval [1] [47]) and compare with three baselines (*i.e.*, CoML [44], Code Interpreter [45], and Open Interpreter [46]). *Pass Rate* is used as the evaluation metric, which divides the number of passed problems by all problems.

The **NL2VIS** task converts natural language to data visualizations based on either Python libraries like `Matplotlib` or visualization grammars like `Vega-Lite`. We use two benchmarks (*i.e.*, nvBench [49] and VisEval [50]) and compare with three baselines (*i.e.*, LIDA [30], Chat2Vis [27], and CoML4VIS [50]) [2]. For nvBench, we use the *EX* metric for evaluation, which measures the equivalence of the generated visualizations with the ground truth based on the presented data values and chart types [52]. For VisEval, we use the *Pass Rate* metric to measure the ratio of valid or legal results divided by all queries, and the *Readability Score* judged by GPT-4V(ision) [53] to measure the overall quality of the generated visualizations [50].

The **NL2Insight** task converts analysis goals to data insights in an end-to-end manner, which requires LLMs' comprehensive problem-solving abilities. We use two benchmarks (*i.e.*, InfiAgent-DABench [48] and InsightBench [26]) and compare with two baselines (*i.e.*, AutoGen [16] and AgentPoirot [26]). For InfiAgent-DABench, we calculate the *Accuracy* of problems with correct answers to all problems. For InsightBench, we use the summary-level *LLaMA-3-Eval* and *ROUGE-1* scores as the evaluation metrics, which measure the alignment of the generated insights against the ground truth based on LLaMA-3 judgment and unigram overlap, respectively [26].

2) Results: As shown in Table I, DataLab achieves comparable performance with the SOTA LLM-based baselines on all four BI tasks. These baselines primarily focus on *single* tasks and exhibit the issues discussed in Section I, such as lacking domain knowledge incorporation or unstructured communication. Specifically, DataLab outperforms all baselines on benchmarks including DS-1000, DSEval, InsightBench, and VisEval, spreading over each critical BI stage. While certain baselines excel in *individual* tasks (*e.g.*, NL2SQL), the primary focus of DataLab is to unify the BI workflow with a *single* LLM-based framework, maintaining satisfactory performance across *various* tasks. This unification is particularly beneficial for real-world scenarios requiring multi-task coordination. Moreover, many agent-based SOTA approaches (*e.g.*, Data Interpreter [9]) that follow the reasoning and acting (ReAct) paradigm [54] can be integrated into our framework through DAG-based agent workflows (Section III). This extensibility further enhances DataLab's practical applicability.

For tasks that require the generation of symbolic languages (*e.g.*, NL2DSCode, NL2VIS), DataLab consistently performs well primarily due to the intermediate DSL specifications generated by our *Domain Knowledge Incorporation* module. Although most research benchmarks lack the necessary information for extracting table/column knowledge, DataLab adopts a meticulously-designed data profiling strategy as an alternative (Section IV-C) to fully utilize the provided data schema, enabling LLMs to correctly identify and associate the semantic relationships between data columns and NL queries, which are crucial to generate high-quality DSLs. Consequently, compared to merely feeding the original pure NL queries, these intermediate DSLs can significantly improve LLM-based agents' performance on generating higher-level languages like SQL queries, Python code, or Vega-Lite specifications, as also shown in previous works [55].

For more complex tasks (*e.g.*, NL2Insight) that typically require multi-step reasoning and/or the collaboration of multiple agents, DataLab also achieves a satisfactory performance. Notably, it outperforms AutoGen, a popular multi-agent framework, by up to 5.06% on DABench and 19.35% on InsightBench. This performance gain can be attributed to two key factors: the agents' improved understanding of the involved datasets due to data profiling and the incorporation

[1] We only evaluate DSEval-LeetCode and -SO due to implementation issues.

[2] As some baselines lack support for NL queries related to multiple data tables, we only evaluate on single-table queries for fair comparison.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE OF DATALAB ON RESEARCH BENCHMARKS

BI Stage	Task	Benchmark	Metric	Method & Performance			
Data Preparation	NL2SQL	Spider [42]	Execution Accuracy	DataLab (Ours) 80.70	DAIL-SQL [25] 83.60	PURPLE [21] **87.80**	CHESS [29] 87.20
		BIRD [43]	Execution Accuracy	DataLab (Ours) 61.33	DAIL-SQL [25] 57.41	PURPLE 68.12	CHESS **68.31**
	NL2DSCode	DS-1000 [4]	Pass Rate	DataLab (Ours) **53.80**	CoML [44] 44.20	Code Interpreter [45] 51.60	Open Interpreter [46] 50.50
		DSEval [47]	Pass Rate	DataLab (Ours) **80.99**	CoML 71.90	Code Interpreter 80.58	Open Interpreter 78.10
Data Analysis	NL2Insight	DABench [48]	Accuracy	DataLab (Ours) 75.10	AutoGen [16] 71.48	AgentPoirot [26] **75.88**	- -
		InsightBench [26]	LLaMA-3-Eval	DataLab (Ours) **0.37**	AutoGen 0.31	AgentPoirot 0.35	- -
			ROUGE-1	DataLab (Ours) 0.33	AutoGen 0.28	AgentPoirot **0.35**	- -
Data Visualization	NL2VIS	nvBench [49]	Execution Accuracy	DataLab (Ours) 53.90	LIDA [30] **54.71**	Chat2Vis [27] 53.83	CoML4VIS [50] 51.12
		VisEval [50]	Pass Rate	DataLab (Ours) **75.99**	LIDA 74.66	Chat2Vis 71.91	CoML4VIS 75.27
			Readability Score	DataLab (Ours) 3.73	LIDA 3.77	Chat2Vis 3.70	CoML4VIS **3.80**

NOTE: The best method is marked in **bold**, while the second-best method is marked with underlines.

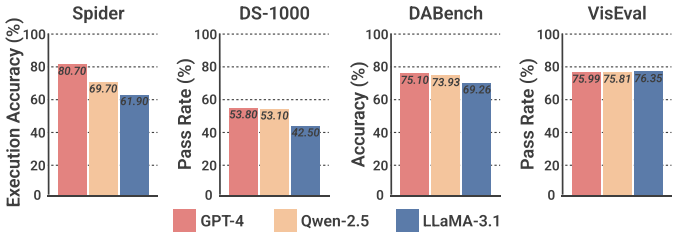


Fig. 6. Performance of DataLab using various underlying LLMs.

of our structured communication mechanism. This mechanism standardizes inter-agent information sharing, enabling a more comprehensive and thorough insight discovery process, especially when provided with high-level analytical objectives.

B. Sensitivity Analysis

To evaluate DataLab's robustness, we experiment with both closed- and open-source LLMs (*i.e.*, **GPT-4**, **Qwen-2.5** [56], and **LLaMA-3.1** [57]) on the above tasks using benchmarks including Spider, DS-1000, DABench, and VisEval.

As shown in Figure 6, DataLab consistently achieves satisfactory performance on all tasks, albeit with some sensitivity to the underlying LLMs. Proprietary models like GPT-4 typically exhibit superior instruction following and code generation abilities, surpassing open-source models like Qwen-2.5 and LLaMA-3.1. For code-intensive tasks like NL2DSCode and NL2Insight, LLaMA-3.1 experiences notable performance drops, especially on DS-1000, due to its relatively weaker code generation capabilities. We further evaluate DS-1000 using vanilla LLaMA-3.1 and achieve a pass rate of 36.90% (lower than 42.50% when integrated with DataLab). Another interesting fact is that, all three LLMs perform similarly on VisEval, with LLaMA-3.1 surprisingly being the best. These findings indicate that DataLab maintains a consistent performance across tasks, despite variations in LLMs, attributed to our data profiling and communication mechanisms. The data profiling mechanism enhances agents' understanding of input data, while the inter-agent communication module enables efficient error handling and iterative refinement, leading to overall performance improvements.

C. Effect of Domain Knowledge Incorporation

1) Knowledge Generation: As described in Section IV-A, this module aims to automatically generate knowledge components of data tables and columns. Deployed at Tencent for one month, **2,426** databases, **262,041** tables, and **2,708,884** columns (averaging 10.3 columns per table) have been successfully processed for knowledge generation, with an average time cost of 45.2 seconds per table. These statistics exhibit the practical application of our approach at a large enterprise.

To evaluate the quality of the generated knowledge, we collect a real-world dataset comprising 50 tables and 629 columns. Each table and column is annotated by domain experts for a ground truth of their semantic meanings. We then compare the *Sentence Embedding Similarity (SES)* between the generated descriptions and the ground truth using M3-Embedding [58], with 1 being identical and 0 being irrelevant. Results show that, the average *SES* scores are **0.712** (60% above 0.7) for tables and **0.677** (53% above 0.7) for columns, indicating the practical utility of the generated knowledge.

Overall, the real-world deployment and quality evaluation demonstrate the efficiency and effectiveness of the knowledge generation process of this module in practical settings.

2) Downstream Tasks: To assess the real-world impact of this module, we evaluate the following downstream tasks:

- The **Schema Linking** task seeks to select relevant tables and columns from the database schema based on NL queries, providing a basis for further analysis [59]. It requires LLMs to precisely capture the semantic relationships between user input and elements of the schema.
- The **NL2DSL** task converts NL queries to DSLs, which have been commonly adopted in commercial BI platforms and are crucial for many downstream tasks [10], [60]. In DataLab, DSLs are used as intermediates for generating SQL queries, Python code, and visualizations.

Due to common issues like ambiguities and jargon in real-world BI scenarios, both tasks require LLMs' deep understanding of domain knowledge. For Schema Linking, we collect a

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

real-world dataset comprising 439 query-table-column pairs, and use *Recall @5* for evaluation. For NL2DSL, we compile another dataset comprising 326 query-DSL pairs, and measure the overall *Accuracy*. We then employ this module to generate knowledge for each involved table and column. For comparison, we design the following three experiment settings:

- **S1 (w/o knowledge)**: This setting provides NL queries along with a brief data schema generated by `Pandas`, but no additional knowledge, serving as a baseline. It is commonly adopted by most existing LLM-based agents.
- **S2 (w/ partial knowledge)**: Compared to S1, this setting additionally provides the generated *description*, *usage*, and *tags* of data tables and columns. It accounts for almost all successful cases in our practical deployment.
- **S3 (w/ all knowledge)**: Compared to S2, this setting further provides all generated knowledge of data tables and columns (see Section IV-A). It accounts for approximately 40% successful cases in our practical deployment.

As shown in Table II, DataLab's performance on both tasks improves significantly when provided with enterprise-specific knowledge. Specifically, the *Recall @5* of Schema Linking increases by **38.47%**, and the *Accuracy* of NL2DSL improves by up to **58.58%**. Even with only partial knowledge (S2), the performance still exhibits a significant increase of 30.77% for Schema Linking and 29.14% for NL2DSL compared to the baseline (S1). During our deployment at Tencent, we observe that many real-world business tables lack sufficient information required for generating comprehensive knowledge, often limited to table and column *descriptions*, *usage*, and *tags*. While understanding the semantic meanings of ambiguous table/column names can largely enhance LLMs' performance, the absence of other knowledge - especially *calculation logic* of derived columns for NL2DSL - can impede their capabilities in certain scenarios. This explains the performance difference between S2 and S3. Despite this, the promising results of S2 guarantee a minimum acceptable level of performance, demonstrating the module's effectiveness and robustness for downstream tasks in real-world BI scenarios.

D. Effect of Inter-Agent Communication

We experiment with a complex BI scenario that involves multiple tasks performed by distinct agents: NL2SQL, NL2DSCode, NL2VIS, Anomaly Detection, Causal Analysis, and Time Series Forecasting. We compile a dataset from practical settings at Tencent, consisting of 2 databases, 10 tables, and 111 columns. For each table, we meticulously design 10 complex questions derived from real-world business queries, totaling to 100 samples. Each question requires multi-step reasoning and multi-agent collaboration, ensuring a rigorous evaluation of our inter-agent communication mechanism.

We evaluate this module's efficiency and effectiveness by respectively calculating the *Success Rate* and *Accuracy* of the agents' responses across all questions. The *Success Rate* measures the ratio of questions that can be successfully solved within a maximum of 5 calls per agent, while the *Accuracy* measures the ratio of correct answers among all questions. For comparison, we employ three experiment settings:

TABLE II
ABLATION STUDY ON DOMAIN KNOWLEDGE INCORPORATION

Task / Metric	S1	S2	S3
Schema Linking / Recall @5 (%)	41.02	71.79	**79.49**
NL2DSL / Accuracy (%)	32.52	61.66	**91.10**

TABLE III
ABLATION STUDY ON INTER-AGENT COMMUNICATION

Metric	S1	S2	S3
Success Rate (%)	73.00	85.00	**92.00**
Accuracy (%)	56.00	79.00	**84.00**

- **S1 (w/o FSM)** [19]: This setting removes the FSM-based information sharing protocol. Therefore, each agent receives *all* information from the shared buffer.
- **S2 (w/o information formatting)** [16]: This setting removes the information format structure and adopts *pure natural language* for inter-agent communication.
- **S3 (w/ both)**: This setting keeps both techniques.

As illustrated in Table III, DataLab's performance on complex BI tasks improves by **19.00%** in *Success Rate* and **28.00%** in *Accuracy* with our inter-agent communication mechanism. Without the FSM-based information sharing protocol (S1), performance significantly degrades. Error analysis reveals that most failures involve more than 3 agents, resulting in overwhelming and irrelevant information that hinders LLMs' reasoning, thereby leading to incorrect outputs [20]. Additionally, the absence of the information format structure (S2) leads to a 7% decrease in *Success Rate* and a 5% drop in *Accuracy*, highlighting the importance of structured prompts in enhancing LLM comprehension and reducing information sharing ambiguities. This is critical in BI scenarios where complex tasks often require iterative error handling for data processing and structured summaries for lengthy outputs.

E. Effect of Cell-based Context Management

1) DAG Construction: To evaluate the efficiency of the DAG construction process, we collect 50 DataLab notebooks containing multi-language cells from practical settings, with cell counts ranging from 2 to 49. We measure the *Time Cost* of DAG construction both at notebook-level and cell-level. The initial construction encompasses all cells upon notebook opening, whereas subsequent updates generally involve a single cell. Our goal is to ensure a reasonable cold-start time while maintaining real-time responsiveness for subsequent updates.

As shown in Figure 7, DAG construction and updating maintain low time costs, at less than **250** and **10 milliseconds**, respectively. The total time for DAG construction increases with cell count, reaching a maximum of 232.22 milliseconds for 35 cells. In contrast, the per-cell time for DAG updating averages to 3.78 milliseconds, peaking at 9.84 milliseconds for 5 cells. Time costs are affected not only by cell count but also by lines of code, accounting for observed fluctuations. Given that a typical DataLab notebook contains fewer than 50 cells, these results demonstrate the efficiency of DAG construction.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

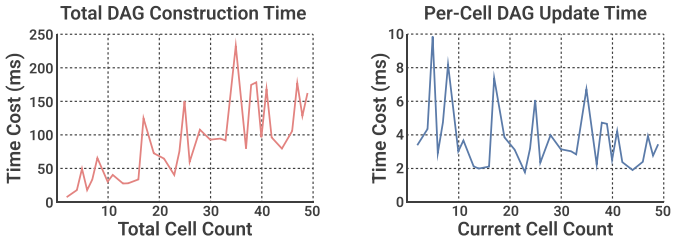


Fig. 7. Time cost of DAG construction.

TABLE IV
ABLATION STUDY ON CELL-BASED CONTEXT MANAGEMENT

Metric	S1	S2
Accuracy (%)	**86.67**	82.00
Token Cost per Query (K)	10.69	**4.10**

2) Task Completion: For each notebook in our collected dataset, we derive 3 real-world user queries, which involve NL2SQL, NL2DSCode, and NL2VIS tasks, totaling to 150 samples. We evaluate this module's performance and cost-effectiveness using two metrics: *Accuracy* and *Token Cost per Query*. For comparison, we conduct an ablation study with two experiment settings: **S1 (w/o DAG)** and **S2 (w/ DAG)**.

As illustrated in Table IV, DataLab achieves a satisfactory *Accuracy* under both settings. Further analysis reveals that certain Markdown cells may contain critical information for task completion, which are occasionally failed to retrieve by our context retrieval mechanism due to limitations of embedding similarity [61]. This accounts for the slight 4.67% drop in *Accuracy* under S2. However, S2 significantly reduces the *Token Cost per Query* by **61.65%** compared to S1, saving approximately $65.90 for every 1,000 queries using GPT-4. This is achieved by identifying the *minimum set* of relevant cells based on DAGs. These results demonstrate this module's cost-effectiveness while maintaining acceptable performance.

VIII. Real-World Applications

DataLab has been effectively deployed on Tencent TEG's Big Data Platform, achieving an average of 2,093 monthly registered users, 10,900 monthly API calls, and 319 weekly active users over a three-month period. It significantly improves the efficiency of data professionals by integrating LLM-powered BI tasks into a unified notebook interface. Below, we highlight three practical use cases to illustrate its benefits.

- Fiona is a data engineer who regularly prepares data for product managers. Previously, she typically adds domain knowledge to prompts manually to enhance performance. With DataLab's *Domain Knowledge Incorporation* module, she is surprised to discover that it can inherently understand her requirements even with ambiguous column names.
- Henry, a data scientist specializing in recommendation algorithms, values DataLab for generating and displaying Python code directly in notebook cells. It makes editing easier and eliminates the need to repetitively copy-paste between chat-based LLM interfaces and traditional notebooks.
- Jerry is a data analyst who provides data reports for stakeholders. Using DataLab, he cuts the time to create data visualizations by 87% (from 15 minutes to 2 minutes). He highly appreciates DataLab's Chart cells, which enable GUI-based customization of automatically generated visualizations.

Additionally, users agree that working on DataLab facilitates communication efficiency with colleagues. For example, they can leave comments next to cells to quickly notify collaborators about their progress, avoiding the need to tediously share screenshots or code snippets as they did previously.

IX. Related Work

Business Intelligence Platforms. BI platforms support users in analyzing business data for decision-making [8], [12]. Representatives like Tableau [62], Power BI [63], and Databricks [64] provide GUIs to support user interactions for data transformation and dashboard generation. These platforms also integrate natural language interfaces [65], [66] to lower the burden of manual operation. Quamar *et al.* [1] proposed an ontology-based method based on business models to provide semantic information and reasoning capability for query interpretation. The emergence of LLMs has further enhanced the domain knowledge integration and visualization generation abilities of BI platforms [67], [68]. Unlike existing tools that focus on visualization (*e.g.*, Tableau, Power BI) or are optimized for engineering workflows (*e.g.*, Databricks), DataLab provides a unified platform to satisfy various BI stages (*i.e.*, data preparation, analysis, and visualization) and data roles (*i.e.*, data engineers, scientists, and analysts) through a one-stop LLM-based agent framework.

LLM-based Data Analysis. LLMs have shown remarkable abilities in semantic understanding and logical reasoning, enabling complex data analysis through conversational interfaces [6], [22], [69]. For example, Table-GPT [70] fine-tunes LLMs on synthesized table-task data to enhance their table-understanding abilities. Chat2Query [23] decomposes NL2SQL tasks into multiple steps to improve generation quality. InsightPilot [71] automates the discovery of data insights and synthesizes them into high-level overviews. Moreover, Chat2Data [15] leverages domain knowledge based on vector databases to mitigate LLMs' hallucination issues, while our work uniquely facilitates domain-specific data analysis by automatically extracting knowledge from enterprise scripts and data lineage information, avoiding manual curation. Unlike existing works that focus on end-to-end results, we introduce a novel notebook interface that supports flexible human intervention - allowing refinement of intermediate SQL, Python, or charts - while unifying tasks fragmented across prior task-specific approaches like LIDA [30] and PURPLE [21].

X. Conclusion

This paper introduces DataLab, a unified BI platform that combines an LLM-based agent framework with a notebook interface. DataLab features a domain knowledge incorporation module, an inter-agent communication mechanism, and a cell-based context management strategy. These components enable seamless integration of LLM assistance with user customization, making DataLab well-suited for practical BI scenarios. DataLab has proven effective on both research benchmarks and real-world business datasets from Tencent.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. Quamar, F. Özcan, D. Miller, R. J. Moore, R. Niehus, and J. T. Kreulen, "Conversational BI: an ontology-driven conversationsystem for business intelligence applications," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3369–3381, 2020.

[2] R. Cao, F. Lei, H. Wu, J. Chen, Y. Fu, H. Gao, X. Xiong, H. Zhang, Y. Mao, W. Hu, T. Xie, H. Xu, D. Zhang, S. Wang, R. Sun, P. Yin, C. Xiong, A. Ni, Q. Liu, V. Zhong, L. Chen, K. Yu, and T. Yu, "Spider2-v: How far are multimodal agents from automating data science and engineering workflows?" *CoRR*, vol. abs/2407.10956, 2024.

[3] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, Z. Liu, Y. Xu, H. SU, D. Shin, C. Xiong, and T. Yu, "Openagents: An open platform for language agents in the wild," in *COLM*, 2024.

[4] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W. Yih, D. Fried, S. I. Wang, and T. Yu, "DS-1000: A natural and reliable benchmark for data science code generation," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 18 319–18 345.

[5] Y. Wu, Y. Wan, H. Zhang, Y. Sui, W. Wei, W. Zhao, G. Xu, and H. Jin, "Automated data visualization from natural language via large language models: An exploratory study," *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 115, 2024.

[6] L. Weng, X. Wang, J. Lu, Y. Feng, Y. Liu, and W. Chen, "Insightlens: Discovering and exploring insights from conversational contexts in large-language-model-powered data analysis," *CoRR*, vol. abs/2404.01644, 2024.

[7] R. Kosara, "Notebooks for data analysis and visualization: Moving beyond the data," *IEEE Computer Graphics and Applications*, vol. 43, no. 1, pp. 91–96, 2023.

[8] V. V. Meduri, A. Quamar, C. Lei, V. Efthymiou, and F. Ozcan, "BI-REC: guided data analysis for conversational business intelligence," *CoRR*, vol. abs/2105.00467, 2021.

[9] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, L. Zhang, M. Yang, M. Zhuge, T. Guo, T. Zhou, W. Tao, W. Wang, X. Tang, X. Lu, X. Zheng, X. Liang, Y. Fei, Y. Cheng, Z. Xu, and C. Wu, "Data interpreter: An LLM agent for data science," *CoRR*, vol. abs/2402.18679, 2024.

[10] A. Su, A. Wang, C. Ye, C. Zhou, G. Zhang, G. Zhu, H. Wang, H. Xu, H. Chen, H. Li, H. Lan, J. Tian, J. Yuan, J. Zhao, J. Zhou, K. Shou, L. Zha, L. Long, L. Li, P. Wu, Q. Zhang, Q. Huang, S. Yang, T. Zhang, W. Ye, W. Zhu, X. Hu, X. Gu, X. Sun, X. Li, Y. Yang, and Z. Xiao, "Tablegpt2: A large multimodal model with tabular data integration," *CoRR*, vol. abs/2411.02059, 2024.

[11] P. B. Chen, F. Wenz, Y. Zhang, M. Kayali, N. Tatbul, M. J. Cafarella, Ç. Demiralp, and M. Stonebraker, "BEAVER: an enterprise benchmark for text-to-sql," *CoRR*, vol. abs/2409.02038, 2024.

[12] J. Lian, X. Liu, Y. Shao, Y. Dong, M. Wang, Z. Wei, T. Wan, M. Dong, and H. Yan, "Chatbi: Towards natural language to complex business intelligence SQL," *CoRR*, vol. abs/2405.00527, 2024.

[13] M. R. J, K. VM, H. Warrier, and Y. Gupta, "Fine tuning LLM for enterprise: Practical guidelines and recommendations," *CoRR*, vol. abs/2404.10779, 2024.

[14] A. Ibrahim, B. Thérien, K. Gupta, M. L. Richter, Q. G. Anthony, E. Belilovsky, T. Lesort, and I. Rish, "Simple and scalable strategies to continually pre-train large language models," *Trans. Mach. Learn. Res.*, vol. 2024, 2024.

[15] X. Zhao, X. Zhou, and G. Li, "Chat2data: An interactive data analysis system with rag, vector databases and llms," *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 4481–4484, 2024.

[16] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "Autogen: Enabling next-gen LLM applications via multi-agent conversation framework," *CoRR*, vol. abs/2308.08155, 2023.

[17] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *ACL*. Association for Computational Linguistics, 2024, pp. 15 174–15 186.

[18] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: communicative agents for "mind" exploration of large language model society," in *NeurIPS*, 2023.

[19] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for A multi-agent collaborative framework," in *ICLR*. OpenReview.net, 2024.

[20] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 31 210–31 227.

[21] T. Ren, Y. Fan, Z. He, R. Huang, J. Dai, C. Huang, Y. Jing, K. Zhang, Y. Yang, and X. S. Wang, "PURPLE: making a large language model a better SQL writer," in *ICDE*. IEEE, 2024, pp. 15–28.

[22] S. Chen, H. Liu, W. Jin, X. Sun, X. Feng, J. Fan, X. Du, and N. Tang, "Chatpipe: Orchestrating data preparation pipelines by optimizing human-chatgpt interactions," in *SIGMOD Conference Companion*. ACM, 2024, pp. 484–487.

[23] J. Zhu, P. Cai, B. Niu, Z. Ni, K. Xu, J. Huang, J. Wan, S. Ma, B. Wang, D. Zhang, L. Tang, and Q. Liu, "Chat2query: A zero-shot automatic exploratory data analysis system with large language models," in *ICDE*. IEEE, 2024, pp. 5429–5432.

[24] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024.

[25] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *Proc. VLDB Endow.*, vol. 17, no. 5, pp. 1132–1145, 2024.

[26] G. Sahu, A. Puri, J. A. Rodriguez, A. Drouin, P. Taslakian, V. Zantedeschi, A. Lacoste, D. Vázquez, N. Chapados, C. Pal, S. Rajeswar, and I. H. Laradji, "Insightbench: Evaluating business analytics agents through multi-step insight generation," *CoRR*, vol. abs/2407.06423, 2024.

[27] P. Maddigan and T. Susnjak, "Chat2vis: Generating data visualizations via natural language using chatgpt, codex and GPT-3 large language models," *IEEE Access*, vol. 11, pp. 45 181–45 193, 2023.

[28] Y. Yu, L. Shen, F. Long, H. Qu, and H. Chen, "Pygwalker: On-the-fly assistant for exploratory visual data analysis," *CoRR*, vol. abs/2406.11637, 2024.

[29] S. Talaei, M. Pourreza, Y. Chang, A. Mirhoseini, and A. Saberi, "CHESS: contextual harnessing for efficient SQL synthesis," *CoRR*, vol. abs/2405.16755, 2024.

[30] V. Dibia, "LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models," in *ACL (demo)*. Association for Computational Linguistics, 2023, pp. 113–126.

[31] M. Tang, S. Shao, W. Yang, Y. Liang, Y. Yu, B. Saha, and D. Hyun, "SAC: A system for big data lineage tracking," in *ICDE*. IEEE, 2019, pp. 1964–1967.

[32] K. Tian, E. Mitchell, A. Zhou, A. Sharma, R. Rafailov, H. Yao, C. Finn, and C. D. Manning, "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in *EMNLP*. Association for Computational Linguistics, 2023, pp. 5433–5442.

[33] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating LLM hallucination via self reflection," in *EMNLP (Findings)*. Association for Computational Linguistics, 2023, pp. 1827–1843.

[34] C. Gormley and Z. Tong, *Elasticsearch: the definitive guide: a distributed real-time search and analytics engine*. " O'Reilly Media, Inc.", 2015.

[35] StarRocks, "Starrocks: A high-performance analytical database," https://www.starrocks.io/, 2024.

[36] D. C. Chiang and H. Lee, "Can large language models be an alternative to human evaluations?" in *ACL*. Association for Computational Linguistics, 2023, pp. 15 607–15 631.

[37] F. Pezoa, J. L. Reutter, F. Suárez, M. Ugarte, and D. Vrgoc, "Foundations of JSON schema," in *WWW*. ACM, 2016, pp. 263–273.

[38] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS*, 2022.

[39] Z. Xi, S. Jin, Y. Zhou, R. Zheng, S. Gao, J. Liu, T. Gui, Q. Zhang, and X. Huang, "Self-polish: Enhance reasoning in large language models via problem refinement," in *EMNLP (Findings)*. Association for Computational Linguistics, 2023, pp. 11 383–11 406.

[40] D. Li, Y. Ma, N. Wang, Z. Ye, Z. Cheng, Y. Tang, Y. Zhang, L. Duan, J. Zuo, C. Yang, and M. Tang, "Mixlora: Enhancing large language models fine-tuning with lora-based mixture of experts," 2024. [Online]. Available: https://arxiv.org/abs/2404.15159

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

[41] K. R. Fall and W. R. Stevens, *Tcp/ip illustrated*. Addison-Wesley Professional, 2012, vol. 1.

[42] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. R. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *EMNLP*. Association for Computational Linguistics, 2018, pp. 3911–3921.

[43] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as A database interface? A big bench for large-scale database grounded text-to-sqls," in *NeurIPS*, 2023.

[44] L. Zhang, Y. Zhang, K. Ren, D. Li, and Y. Yang, "Mlcopilot: Unleashing the power of large language models in solving machine learning tasks," in *EACL*. Association for Computational Linguistics, 2024, pp. 2931–2959.

[45] shroominic, "Open source implementation of the chatgpt code interpreter," https://github.com/shroominic/codeinterpreter-api, [Accessed 18-10-2024].

[46] KillianLucas, "A natural language interface for computers," https://github.com/OpenInterpreter/open-interpreter, [Accessed 13-02-2025].

[47] Y. Zhang, Q. Jiang, X. XingyuHan, N. Chen, Y. Yang, and K. Ren, "Benchmarking data science agents," in *ACL*. Association for Computational Linguistics, 2024, pp. 5677–5700.

[48] X. Hu, Z. Zhao, S. Wei, Z. Chai, Q. Ma, G. Wang, X. Wang, J. Su, J. Xu, M. Zhu, Y. Cheng, J. Yuan, J. Li, K. Kuang, Y. Yang, H. Yang, and F. Wu, "Infiagent-dabench: Evaluating agents on data analysis tasks," in *ICML*. OpenReview.net, 2024.

[49] Y. Luo, N. Tang, G. Li, C. Chai, W. Li, and X. Qin, "Synthesizing natural language to visualization (NL2VIS) benchmarks from NL2SQL benchmarks," in *SIGMOD Conference*. ACM, 2021, pp. 1235–1247.

[50] N. Chen, Y. Zhang, J. Xu, K. Ren, and Y. Yang, "Viseval: A benchmark for data visualization in the era of large language models," *IEEE Transactions on Visualization and Computer Graphics*, 2024.

[51] OpenAI, "GPT-4 technical report," *CoRR*, vol. abs/2303.08774, 2023.

[52] G. Li, X. Wang, G. Aodeng, S. Zheng, Y. Zhang, C. Ou, S. Wang, and C. H. Liu, "Visualization generation with large language models: An evaluation," *CoRR*, vol. abs/2401.11255, 2024.

[53] Z. Yang, L. Li, K. Lin, J. Wang, C. Lin, Z. Liu, and L. Wang, "The dawn of lmms: Preliminary explorations with gpt-4v(ision)," *CoRR*, vol. abs/2309.17421, 2023.

[54] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *ICLR*. OpenReview.net, 2023.

[55] B. Wang, Z. Wang, X. Wang, Y. Cao, R. A. Saurous, and Y. Kim, "Grammar prompting for domain-specific language generation with large language models," in *NeurIPS*, 2023.

[56] Q. Team, "Qwen2.5: A party of foundation models," September 2024. [Online]. Available: https://qwenlm.github.io/blog/qwen2.5/

[57] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Rozière, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. M. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, and et al., "The llama 3 herd of models," *CoRR*, vol. abs/2407.21783, 2024.

[58] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *ACL (Findings)*. Association for Computational Linguistics, 2024, pp. 2318–2335.

[59] W. Lei, W. Wang, Z. Ma, T. Gan, W. Lu, M. Kan, and T. Chua, "Re-examining the role of schema linking in text-to-sql," in *EMNLP*. Association for Computational Linguistics, 2020, pp. 6943–6954.

[60] A. Popovic, I. Lukovic, V. Dimitrieski, and V. Djukic, "A DSL for modeling application-specific functionalities of business applications," *Comput. Lang. Syst. Struct.*, vol. 43, pp. 69–95, 2015.

[61] H. Steck, C. Ekanadham, and N. Kallus, "Is cosine-similarity of embeddings really about similarity?" in *WWW (Companion Volume)*. ACM, 2024, pp. 887–890.

[62] Tableau, "Tableau einstein," https://www.tableau.com/, 2024.

[63] Microsoft, "Power bi," https://www.microsoft.com/en-us/power-platform/products/power-bi, 2024.

[64] Databricks, "Databricks data intelligence platform," https://www.databricks.com/, 2024.

[65] M. Tory and V. Setlur, "Do what I mean, not what I say! design considerations for supporting intent and context in analytical conversation," in *VAST*. IEEE, 2019, pp. 93–103.

[66] Y. Feng, X. Wang, B. Pan, K. Wong, Y. Ren, S. Liu, Z. Yan, Y. Ma, H. Qu, and W. Chen, "XNLI: explaining and diagnosing nli-based visual data analysis," *IEEE Trans. Vis. Comput. Graph.*, vol. 30, no. 7, pp. 3813–3827, 2024.

[67] X. Miao, Z. Jia, and B. Cui, "Demystifying data management for large language models," in *SIGMOD Conference Companion*. ACM, 2024, pp. 547–555.

[68] X. Zhou, X. Zhao, and G. Li, "Llm-enhanced data management," *CoRR*, vol. abs/2402.02643, 2024.

[69] B. Chopra, A. Singha, A. Fariha, S. Gulwani, C. Parnin, A. Tiwari, and A. Z. Henley, "Conversational challenges in ai-powered data science: Obstacles, needs, and design opportunities," *CoRR*, vol. abs/2310.16164, 2023.

[70] P. Li, Y. He, D. Yashar, W. Cui, S. Ge, H. Zhang, D. R. Fainman, D. Zhang, and S. Chaudhuri, "Table-gpt: Table fine-tuned GPT for diverse table tasks," *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 176, 2024.

[71] P. Ma, R. Ding, S. Wang, S. Han, and D. Zhang, "Insightpilot: An llm-empowered automated data exploration system," in *EMNLP (Demos)*. Association for Computational Linguistics, 2023, pp. 346–352.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.